

# 7 HACKS

## para ahorrar tokens en Claude

---

Deja de quedarte sin mensajes antes de tiempo.

### 1 El Método Cavernícola

~40% tokens por respuesta

### 2 Code Review Graph

60-70% en código

### 3 Elige el modelo correcto

hasta 5x de ahorro

### 4 No subas PDFs directo

80% por sesión

### 5 El Truco de Proyectos

sin repetir contexto

### 6 Skill Compacto

sin historial inflado

### 7 Evita las horas pico

más velocidad

# Por qué los tokens importan

Cada vez que escribes algo en Claude y Claude te responde, esas palabras se cuentan como "tokens". Piensa en los tokens como el combustible de tu conversación.

Cuanto más tokens usa una conversación, más rápido llegas a tu límite de mensajes. Estos 7 hacks te ayudan a usar menos tokens y avanzar más antes de chocar con ese límite.

---

## Para quién es esta guía

Usuarios de Claude Pro, Max y API que sienten que se quedan sin mensajes demasiado rápido. Si tu contexto se corta antes de terminar una tarea o ves el límite de uso antes de lo que quisieras, esta guía es para ti.

# El Método "Cavernícola"

Haz que Claude hable directo. Sin relleno. Sin rodeos.

## QUÉ HACER

Claude por defecto agrega mucho relleno a sus respuestas: "¡Con gusto te ayudo!" o "Por supuesto, la mejor manera de abordarlo sería...". Eso suena bien pero desperdicia tokens. El fix: pega este prompt al inicio de cualquier conversación.

## PROMPT EXACTO

"De ahora en adelante, elimina todas las palabras de relleno. Sin "el", "es", "son". Sin cortesías ni preámbulos. Solo respuesta directa. Oraciones cortas de 3-6 palabras. Ejecuta primero, muestra el resultado, luego para. No narres. Ejemplo: En vez de 'La solución es usar sync', di 'Usa sync'."

## POR QUÉ EN ESTE ORDEN

Ahorra ~40% de tokens por respuesta. La misma información, sin el relleno.

**TIEMPO:** 2 min

- Úsalo siempre que tengas una sesión larga de trabajo. Es el hack con retorno más inmediato.

## Mira la diferencia:

### SIN CAVERNÍCOLA [127 tokens]

*"¡Con gusto te ayudo con eso!  
La mejor manera de hacerlo  
sería implementar una función  
async. También asegúrate de  
manejar los errores..."*

### CON CAVERNÍCOLA [32 tokens]

*"Usa función async.  
Agrega bloque try-catch.  
Maneja errores con  
.catch() o await."*

**Misma info. 75% menos tokens.**

## HACK #2

# Usa Code Review Graph

Deja de hacer que Claude lea todo tu código una y otra vez.

### QUÉ HACER

Si compartes código con Claude, lee cada línea cada vez que mandas un mensaje. En proyectos grandes esto quema tokens muy rápido. Code Review Graph convierte tu código en un resumen estructurado: Claude lee el mapa, no cada archivo.

→ [github.com/tirth8205/code-review-graph](https://github.com/tirth8205/code-review-graph)

# Elige el Modelo Correcto

Opus consume 5x más tokens que Sonnet. La mayoría de tareas no necesitan Opus.

## QUÉ HACER

Claude viene en diferentes modelos. Usar el equivocado es el error más costoso y más silencioso.

### USA SONNET PARA:

- Implementación de código
- Análisis de datos
- Preguntas generales
- Resúmenes de documentos

### USA OPUS PARA:

- Decisiones de arquitectura compleja
- Diagnóstico de bugs críticos
- Escritura compleja y de alto impacto

### USA HAIKU PARA:

- Respuestas rápidas
- Tareas simples del día a día
- Solicitudes no técnicas

## POR QUÉ EN ESTE ORDEN

- Regla simple: Sonnet por defecto. Opus solo cuando necesitas razonamiento profundo.

# No Subas PDFs Directo a Claude

Un PDF sin procesar puede consumir el 80% de tu sesión en un solo mensaje.

## QUÉ HACER

Los PDFs tienen mucho formato oculto, espacios en blanco y texto de relleno que cuentan como tokens. En lugar de subirlos directo, sigue estos 4 pasos:

- 1 Sube el PDF a ChatGPT primero
- 2 Usa este prompt: "Lee este documento. Elimina palabras de relleno, formato extra y texto redundante. Devuelve solo la información clave como texto plano."
- 3 Copia el resultado condensado de ChatGPT
- 4 Pégallo en Claude

## POR QUÉ EN ESTE ORDEN

Le das a Claude la misma información, pero en una fracción de los tokens.

**TIEMPO:** 5 min por documento

- Para documentos largos este paso solo te ahorra frustraciones. Un PDF de 50 páginas puede cortar tu sesión a la mitad.

# El Truco de los Proyectos

No uses chats normales para trabajo que se repite.

## QUÉ HACER

Cada chat nuevo empieza de cero. Eso significa que cada vez que abres uno, tienes que reexplicar quién eres, qué haces y cómo trabajas. Eso son tokens desperdiciados antes de siquiera empezar.

- 1 Crea un Proyecto en Claude.ai
- 2 Pon tus instrucciones permanentes en el system prompt del proyecto
- 3 Trabaja siempre desde ahí, no desde chats sueltos

## POR QUÉ EN ESTE ORDEN

El contexto no se repite. Las instrucciones viven en un solo lugar. Claude ya sabe quién eres desde el mensaje 1, sin gastar ni un token en presentaciones.

**TIEMPO:** 10 min de configuración inicial

- Buenos nombres de proyecto: "Mi Negocio", "Contenido Redes", "Clientes". Evita nombres genéricos como "Trabajo" o "General".

# Crea un Skill "Compacto"

Los chats largos consumen tokens exponencialmente.

## QUÉ HACER

Mientras más larga se vuelve una conversación, más tokens consume cada nuevo mensaje, porque Claude relee todo el historial cada vez. Pero abrir un chat nuevo significa perder todo el contexto. El fix: el skill COMPACTO.

## PROMPT EXACTO

"Crea este skill: Cuando diga COMPACTO, resume toda nuestra conversación en 5-7 puntos clave con todo el contexto importante, decisiones y fragmentos de código. Formato listo para copiar y pegar en un chat nuevo."

## Cómo usarlo cuando el chat se vuelve largo:

- 1 Escribe COMPACTO
- 2 Copia el resumen
- 3 Abre chat nuevo y pégalo como primer mensaje

## POR QUÉ EN ESTE ORDEN

Todo el contexto, sin el historial inflado. Empiezas fresco sin perder nada.

**TIEMPO:** 2 min de configuración

- Úsalo cuando el chat supere los 20 mensajes o sientas que las respuestas se vuelven menos precisas.



# Evita las Horas Pico

Durante las horas de mayor tráfico, Claude responde más lento y los límites se sienten más ajustados.

## QUÉ HACER

Planea tu trabajo intenso con Claude fuera de los horarios de mayor uso de servidores.

### PEORES HORAS:

- Lun-Vie 5am-11am PT
- 1pm-7pm GMT

### MEJORES HORAS:

- Fines de semana
- Tardes entre semana
- Noche / madrugada

## POR QUÉ EN ESTE ORDEN

No cambia cuántos tokens tienes, pero sí qué tan rápido y fluido es usarlos. En horas de baja carga el trabajo fluye mejor.

**TIEMPO:** 0 min — solo ajustar tu horario

- Si estás en LATAM, la noche (después de las 9pm hora local) suele ser el horario más tranquilo para usar Claude.

# Ya tienes los 7 hacks.

Aplicálos en orden y ve cuáles cambian más tu flujo de trabajo.  
Cualquier duda, déjala en el hilo de esta sección en The Grid.

## Resumen rápido

|    |                          |                                 |
|----|--------------------------|---------------------------------|
| #1 | Método Cavernícola       | ~40% menos tokens por respuesta |
| #2 | Code Review Graph        | 60-70% en proyectos de código   |
| #3 | Elige el modelo correcto | hasta 5x según el modelo        |
| #4 | No subas PDFs directo    | hasta 80% por sesión            |
| #5 | Truco de Proyectos       | sin repetir contexto            |
| #6 | Skill Compacto           | sin historial inflado           |
| #7 | Evita horas pico         | más velocidad y margen          |

### ¿Quieres aprender IA gratis?

Únete a The Grid en Skool. Guías, recursos y comunidad  
para aprender IA sin gastar un peso.

[skool.com/the-grid-5633](https://skool.com/the-grid-5633)